TECNOLÓLICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE ENERO – AGOSTO 2026

CARRERA

Ing. Sistemas computacionales

MATERIA

Programaci[on

TÍTULO

Abstract Factory

UNIDAD A EVALUAR

Unidad 2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO

Castillejo Robles Lennyn Alejandro 22210880

NOMBRE DEL MAESTRO

Maribel Guerrero Luis

FECHA DE ENTREGA

05 de marzo del 2026

Introducción al patrón Abstract Factory

El patrón de diseño Abstract Factory pertenece a la categoría de patrones creacionales y tiene como objetivo proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.

Este patrón permite separar la lógica de creación de objetos del resto del sistema, evitando que el código cliente dependa directamente de clases específicas. En lugar de instanciar objetos utilizando new en diferentes partes del programa, se define una fábrica abstracta que declara métodos para crear los distintos tipos de productos.

Posteriormente, cada fábrica concreta implementa esos métodos para crear una combinación específica de objetos que pertenecen a la misma familia.

La familia de objetos está formada por los siguientes elementos:

- Guia
- Examen

Las fábricas concretas representan diferentes modalidades educativas, las cuales generan materiales coherentes con su modalidad:

- MaterialPresencialFactory
- MaterialVirtualFactory
- MaterialHibridoFactory

Cada fábrica garantiza que los objetos creados (guía y examen) pertenezcan a la misma modalidad educativa, manteniendo consistencia en el sistema.

Estructura general de la implementación

La estructura principal del patrón se encuentra definida en el archivo AbstractFactory.cs, donde se declaran las clases y métodos base que serán utilizados por las implementaciones concretas.

Fábrica abstracta

La clase MaterialFactory define los métodos que deberán implementar todas las fábricas concretas:

- **CrearGuia()**
- **CrearExamen()**

Esta clase funciona como un contrato, asegurando que todas las fábricas produzcan los mismos tipos de objetos.

Productos abstractos

También se definen los productos abstractos del sistema:

Guia

- Contiene el método Mostrar() que representa la forma en que se presenta la guía al estudiante.

Examen

- Contiene el método Aplicar() que representa la forma en que se realiza el examen.

Estas clases sirven como base para todas las implementaciones concretas.

Fábricas concretas

Las fábricas concretas son responsables de crear objetos específicos según la modalidad educativa.

MaterialPresencialFactory

- Crea GuiaImpresa
- Crea ExamenEnPapel

MaterialVirtualFactory

- Crea GuiaPDF
- Crea ExamenOnline

MaterialHibridoFactory

- Crea GuiaSemi
- Crea ExamenSemiPresencial

Cada fábrica garantiza que los objetos producidos pertenezcan a la misma familia de productos.

Cliente

El cliente, ubicado en Program.Main, interactúa únicamente con las clases abstractas (MaterialFactory, Guia, Examen). Esto permite cambiar el tipo de material generado simplemente cambiando la fábrica utilizada.

Código implementado

El flujo principal del programa consiste en seleccionar una fábrica concreta, crear los productos correspondientes y ejecutar sus métodos.

Pasos del flujo:

1. Se selecciona una fábrica concreta.
2. La fábrica crea los objetos correspondientes.
3. Se utilizan los métodos definidos en los productos.
4. El proceso puede repetirse con otras fábricas.

Código utilizado

```
1   using System;
2   using System.Collections.Generic;
3   using System.Linq;
4   using System.Text;
5   using System.Threading.Tasks;
6
7   namespace AbstractFactory
8   {
9       public abstract class MaterialFactory
10      {
11          public abstract Guia CrearGuia();
12
13          public abstract Examen CrearExamen();
14      }
15
16      public abstract class Guia
17      {
18          public abstract void Mostrar();
19      }
20
21      public abstract class Examen
22      {
23          public abstract void Aplicar();
24      }
25      }
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace AbstractFactory
8  {
9      public class MaterialHibridoFactory : MaterialFactory
10     {
11         public override Guia CrearGuia()
12         {
13             return new GuiaSemi();
14         }
15         public override Examen CrearExamen()
16         {
17             return new ExamenSemiPresencial();
18         }
19     }
20 }
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace AbstractFactory
8  {
9      public class GuiaImpresa : Guia
10     {
11         public override void Mostrar()
12         {
13             Console.WriteLine("Mostar la guia impresa");
14         }
15     }
16
17     public class ExamenEnPapel : Examen
18     {
19         public override void Aplicar()
20         {
21             Console.WriteLine("Se aplica Examen en papel");
22         }
23
24     }
25 }
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace AbstractFactory
8  {
9      public class GuiaPDF : Guia
10     {
11         public override void Mostrar()
12         {
13             Console.WriteLine("Monstrar la guia Pdf");
14         }
15     }
16
17     public class ExamenOnline : Examen
18     {
19        public override void Aplicar()
20         {
21             Console.WriteLine("Se aplica examen en linea");
22         }
23     }
24
25 }
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace AbstractFactory
8  {
9      internal class Program
10     {
11         static void Main(string[] args)
12         {
13             MaterialFactory fabrica;
14             fabrica = new MaterialPresencialFactory();
15
16             Guia guia = fabrica.CrearGuia();
17             Examen examen = fabrica.CrearExamen();
18
19             guia.Mostrar();
20             examen.Aplicar();
21
22             Console.WriteLine("");
23
24             fabrica = new MaterialVirtualFactory();
25
26             guia = fabrica.CrearGuia();
27             examen = fabrica.CrearExamen();
28
29             guia.Mostrar();
30             examen.Aplicar();
31             Console.WriteLine("");
32
33
34             fabrica = new MaterialHibridoFactory();
35             guia = fabrica.CrearGuia();
36             examen = fabrica.CrearExamen();
37
38             guia.Mostrar();
39             examen.Aplicar();
40
41             Console.ReadKey();
42         }
43     }
44 }
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace AbstractFactory
8  {
9      public class MaterialPresencialFactory : MaterialFactory
10     {
11         public override Guia CrearGuia()
12         {
13             return new GuiaImpresa();
14         }
15         public override Examen CrearExamen()
16         {
17             return new ExamenEnPapel();
18         }
19     }
20 }
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace AbstractFactory
8  {
9      public class MaterialVirtualFactory : MaterialFactory
10     {
11         public override Guia CrearGuia()
12         {
13             return new GuiaPDF();
14         }
15         public override Examen CrearExamen()
16         {
17             return new ExamenOnline();
18         }
19     }
20 }
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace AbstractFactory
{
    public class GuiaSemi : Guia
    {
        public override void Mostrar()
        {
            Console.WriteLine("Mostar la guia de forma hibrida");
        }
    }

    public class ExamenSemiPresencial : Examen
    {
        public override void Aplicar()
        {
            Console.WriteLine("Se aplica examen en casa o en linea");
        }
    }
}
```

Conclusión del patrón y de la implementación realizada

La implementación realizada permitió comprobar cómo funciona el patrón de diseño Abstract Factory, que ayuda a organizar mejor la creación de objetos dentro del sistema, y en lugar de crear directamente los objetos, se utilizan fabricas que se encargan de generarlos.

Una de las ventajas es que el sistema puede ampliarse fácilmente, por ejemplo si en el futuro se quisiera agregar una nueva modalidad educativa, como sea que implementen esa modalidad solo sería necesario crear nuevas clases y una nueva fábrica sin tener que modificar la estructura principal del programa y asu este mas ordenado, flexible y fácil de poner la modificación deseada.